INFORMATICS INSTITUTE OF TECHNOLOGY

Foundation in Higher Education

Unit Code and Description: DOC325 - Digital Circuits and Logic Design

Lecturer: Ms. Aathika Salam / Ms. Chathuri Udagedara

Assignment Number: 01 **Assignment Type**: Group

Group ID : I4

Issue Date: 7 th March 2023

Submission date & time: on or before 26th of March 2023,11.55 pm.

Marks Weightage: Documentation 40% and Viva 60%

Group Members:

Name	**Student ID**
W.K.A.Madusha	20221044
Azra Safrullah	20221041
Esitha Jayasundara	20221034
J.M.N.P.Dasanayake	20221125
Thedas Gunawardhana	20221097

i. Acknowledgement.

We would like to express our gratitude to all those who gave us the possibility to complete this report. We would also like to acknowledge our module leader, Ms. Aathika Salam and Ms. Chathuri Udagedara for the constant guidance and support she gave us throughout this project. Without Ms. Aathika Salam and the other respective Digital Circuit and Logic Design lecturers, we would not have acquired the necessary knowledge to complete this document; hence, we would like to dedicate a small portion of this text to them. We also want to express our gratitude to all the other IIT lecturers who inspired us and helped us along the way. Finally, we want to extend our gratitude to our parents and friends for helping us in our endeavors.

Contents

1. Question 1

1.1. Introduction and the description of the problem

The potential of theft or burglary is one issue that a bank vault system might encounter in the real world. Although bank vaults are made to safeguard priceless valuables like cash, critical documents, and other precious items, thieves may still attempt to steal these items from them. Strong security measures must be in place for bank vault systems to prevent unauthorized access in order to solve this issue. Bank vault systems must be able to promptly react to any attempted breaches in addition to preventing theft. The bank's vault system is essential to guarantee the protection and safety of its customers and the bank's valuable assets. We have come up with a bank vault system for the above-mentioned security problem. To open or have access to the bank vault, we have 3 security barriers namely the fingerprint, password, and FaceID. All the fingerprints, passwords, and FaceID should match/correct. Thereafter, once you press the 'enter' button, the bank vault will open and the alarm would not ring. But if any of the input does not match/wrong, it will buzz an alarm and the vault would not open. All the conditions should be met for the vault to open. By this we can make sure that only authorized personals have access to the bank vault system.

1.2. Description of the inputs and the outputs

Inputs

- A (fingerprint detecting sensor) = 0 – Invalid fingerprint
  1 – Valid fingerprint
- B (Password) = 0 – Wrong password
  1 – Correct password
- C (face scanner) = 0 – Invalid face id
  1 – Valid face id
- D (Enter button) = 0 – Enter button not pressed
  1 – Enter button pressed

Outputs

- X = 0 – Alarm is off
  1 – Alarm is on

- Y = 0 – Vault is locked

  1 – Vault is unlocked

1.3. Truth table for the above problem

A	B	C	D	A.B	B.C	A.B.B.C	A.B.B.C.D (Y)	A.B.B.C.D⊕D (X)	Minterm	Maxterm
0	0	0	0	0	0	0	0	0	A'.B'.C'.D'	A+B+C+D
0	0	0	1	0	0	0	0	1	A'.B'.C'.D	A+B+C+D'
0	0	1	0	0	0	0	0	0	A'.B'.C .D'	A+B+C'+D
0	0	1	1	0	0	0	0	1	A'.B'.C .D	A+B+C'+D'
0	1	0	0	0	0	0	0	0	A'.B .C'.D'	A+B'+C+D
0	1	0	1	0	0	0	0	1	A'.B .C'.D	A+B'+C+D'
0	1	1	0	0	1	0	0	0	A'.B .C .D'	A+B'+C'+D
0	1	1	1	0	1	0	0	1	A'.B .C .D	A+B'+C'+D'
1	0	0	0	0	0	0	0	0	A .B'.C'.D'	A'+B+C+D
1	0	0	1	0	0	0	0	1	A .B'.C'.D	A'+B+C+D'
1	0	1	0	0	0	0	0	0	A .B'.C .D'	A'+B+C'+D
1	0	1	1	0	0	0	0	1	A .B'.C .D	A'+B+C'+D'

1	1	0	0	1	0	0	0	0	A .B .C'.D'	A'+B'+C+D
1	1	0	1	1	0	0	0	1	A .B .C'.D	A'+B'+C+D'
1	1	1	0	1	1	1	0	0	A .B .C .D'	A'+B'+C'+D
1	1	1	1	1	1	1	1	0	A .B .C .D	A'+B'+C'+D ,

1.4. SOP and POS for the outputs

Y Output

- SOP - (A.B.C.D)

- POS - (A+B+C+D). (A+B+C+D'). (A+B+C'+D). (A+B+C'+D'). (A+B'+C+D). (A+B'+C+D'). (A+B'+C'+D). (A+B'+C'+D'). (A'+B+C+D). (A'+B+C+D'). (A'+B+C'+D). (A'+B+C'+D'). (A'+B'+C+D). (A'+B'+C+D'). (A'+B'+C'+D)

X Output

- SOP – (A'.B'.C'.D)+(A'.B'.C .D)+(A'.B .C'.D)+(A'.B .C .D)+(A .B'.C'.D)+(A .B'.C .D)+(A .B .C'.D)

Table 1truth table

- POS –
  (A+B+C+D).(A+B+C'+D).(A+B'+C+D).(A+B'+C'+D).(A'+B+C+D).(A'+B+C'+D)
  .(A'+B'+C+D).(A'+B'+C'+D).(A'+B'+C'+D')

1.5. Simplified answer using k-map and Boolean laws

1.5.1. Using k-map

K-Map (X)

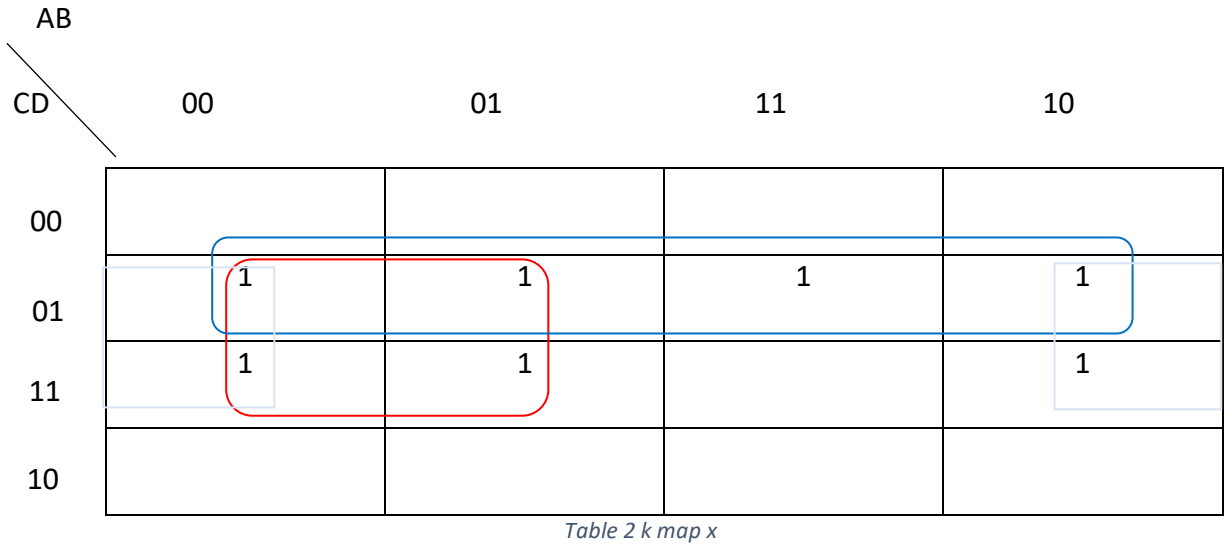

Table 2 k map x

Simplified answer = (A'.D) + (B'.D) + (C'.D)

K-Map (Y)

AB / CD	00	01	11	10
00				
01				
11			1	
10				

Table 3 k map y

Simplified answer = A.B.C.D

1.5.2. Using Boolean laws

SOP(X)=

=(A'.B'.C'.D)+(A'.B'.C.D)+(A'.B.C'.D)+(A'.B.C.D)+(A.B'.C'.D)+(A.B'.C.D)+(A.B.C'.D)

=[(A'+B'+C')D]+(A'.B'.C.D)+(A'.B.C'.D)+(A'.B.C.D)+(A.B'.C'.D)+(A.B'.C.D)+(A.B.C'.D)[By De morgan Law]

=[(A'+B'+C')D]+[(A'+B')C.D]+(A'.B.C'.D)+(A'.B.C.D)+(A.B'.C'.D)+(A.B'.C.D)+(A.B.C'.D) [By De morgan Law]

=[(A'+B'+C')D]+[(A'+B')C.D]+[A'.B.D(C'+C)]+[A(B'+C')D]+(A.B'.C.D)+(A.B.C'.D)

[By Distributive law]

=[(A'+B'+C')D]+[(A'+B')C.D]+[A'.B.D.1]+[A(B'+C')D]+(A.B'.C.D)+(A.B.C'.D)

[By Complement law]

=[(A'+B'+C')D]+[(A'+B')C.D]+(A'.B.D)+[A(B'+C')D]+(A.B'.C.D)+(A.B.C'.D)

[By Identity law]

=[(A'+B'+C')D]+[(A'+B')C.D]+[A.(B'+C')D]+(A.B'.C.D)+[B.D(A.C'+A')] [By Distributive law]

=[(A’+B’+C’)D]+[(A’+B’)C.D]+[A.(B’+C’)D]+(A.B’.C.D)+[B.D(C’+A’)] [By Absorption law]

=(D.A’)+(D.B’)+(D.C’)+[(A’+B’)C.D]+[A.(B’+C’)D]+(A.B’.C.D)+[B.D(C’+A’)] [By Distribution law]

=(D.A’)+(D.B’)+(D.C’)+[(A’+B’)C.D]+[A.(B’+C’)D]+[B.D(C’+A’)] [By Absorption law]

=(D.A’)+(D.B’)+[D(A’+B’)C+C’]+[A(B’+C’)D]+[B.D(C’+A’)] [By Distributive law]

=(D.A’)+(D.B’)+[D(A’+B’+C’)]+[A(B’+C’)D]+[B.D(C’+A’)] [By Absorption law]

=(D.B’)+[D(A’+B’+C’)]+{D[A(B’+C’)+A’]}+[B.D(C’+A’)] [By Distributive law]

=(D.B’)+[D(A’+B’+C’)]+[D(B’+C’+A’)]+[B.D(C’+A’)] [By Absorption law]

=(D.B’)+[D(A’+B’+C’)]+[B.D(C’+A’)] [By Idempotent law]

=[D(A’+B’+C’)]+{D[B(C’+A’)+B’]} [By Distributive law]

=[D(A’+B’+C’)]+[D(C’+A’+B’)] [By Absorption law]

=[D(A’+B’+C’)] [By Idempotent law]

=DA’+DB’+DC’ [By Distribution law]

D.(A’+B’+C’) [By Distributive law]

(ABC)’.D [By De morgans law]

SOP(Y) =

=A.B.B.C.D

=A.B.C.D [By Idempotent law]

X=(ABC)’D

Y=A.B.C.D

1.6. Logic circuit for the simplified epression

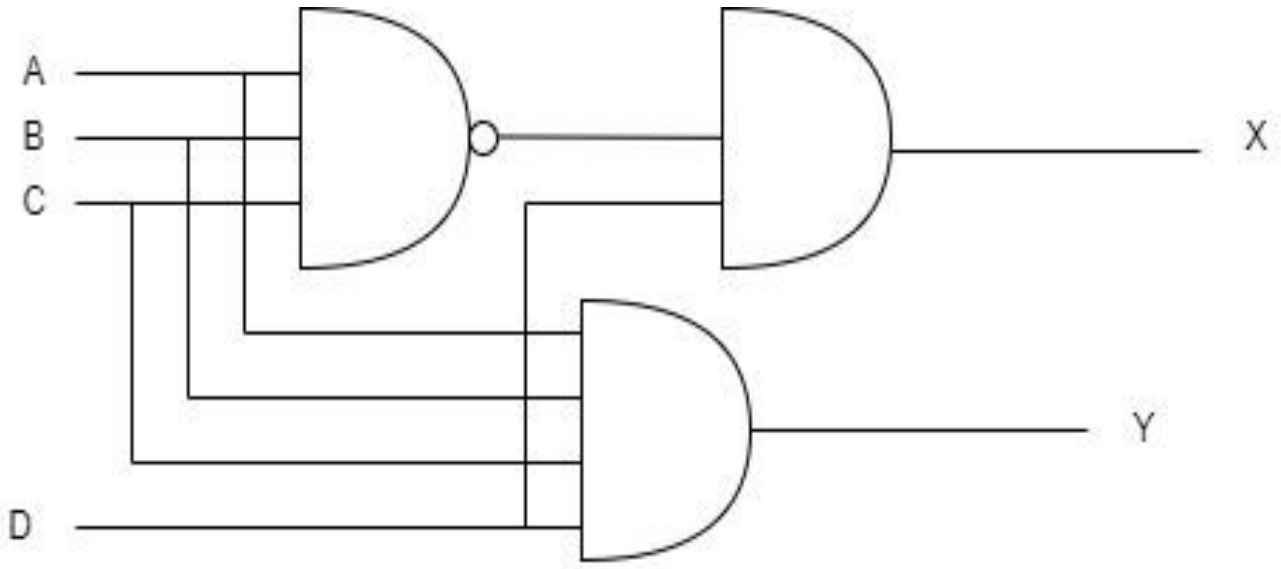

Figure 1logic circuit

1.7. Simulated logic circuit

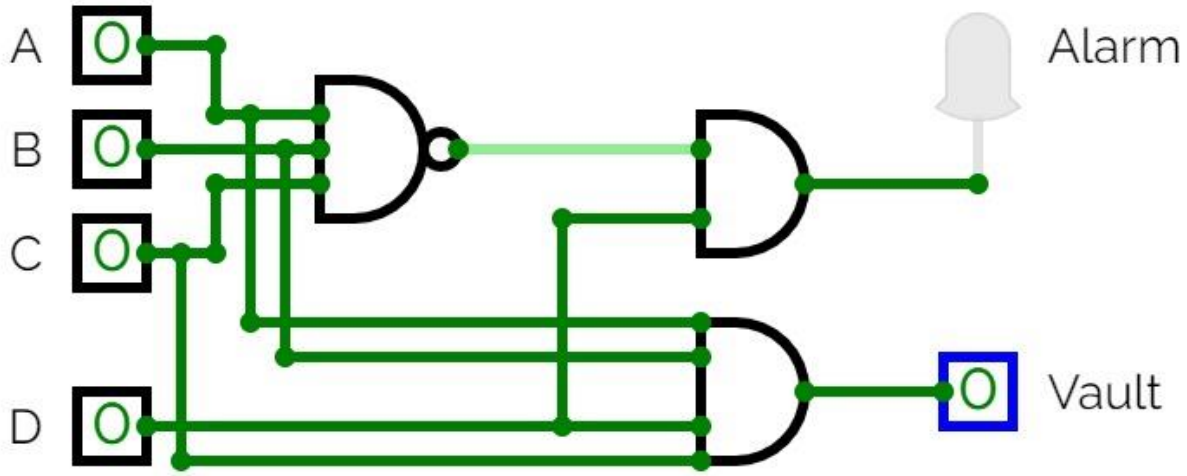

Figure 2simulated logic circuit

2. Question 2

2.1. Schematic diagrams for encoder and decoder and truth tables

2.1.1. Schematic diagram for the 16x4 encoder.

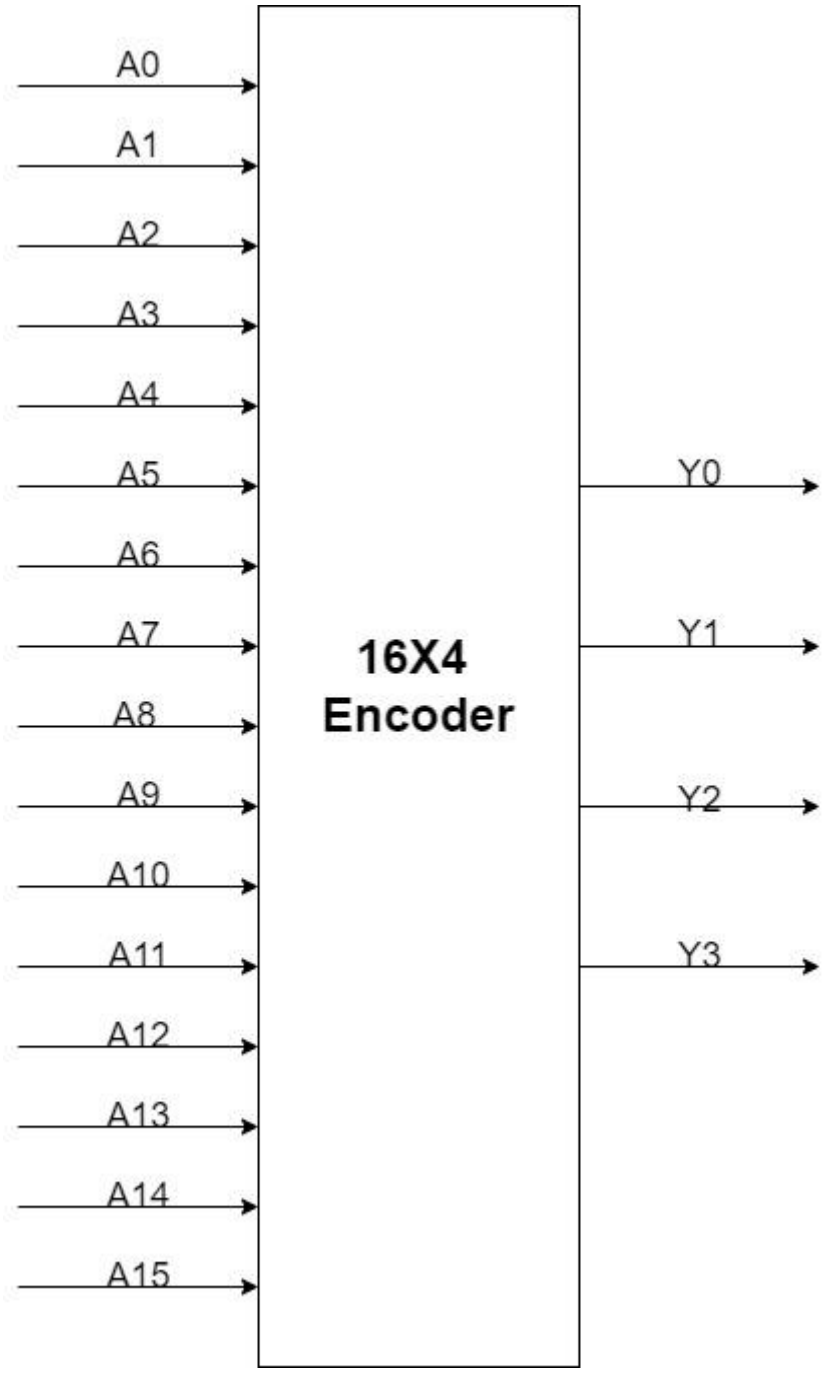

Figure 3 encoder

2.1.2. Truth table for 16X4 encoder

Y_{15}	Y_{14}	Y_{13}	Y_{12}	Y_{11}	Y_{10}	Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	A_3	A_2	A_1	A_0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	1
0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	1	0
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

Table 4 encoder truth table

2.1.3. Schematic diagram for 16X4 decoder

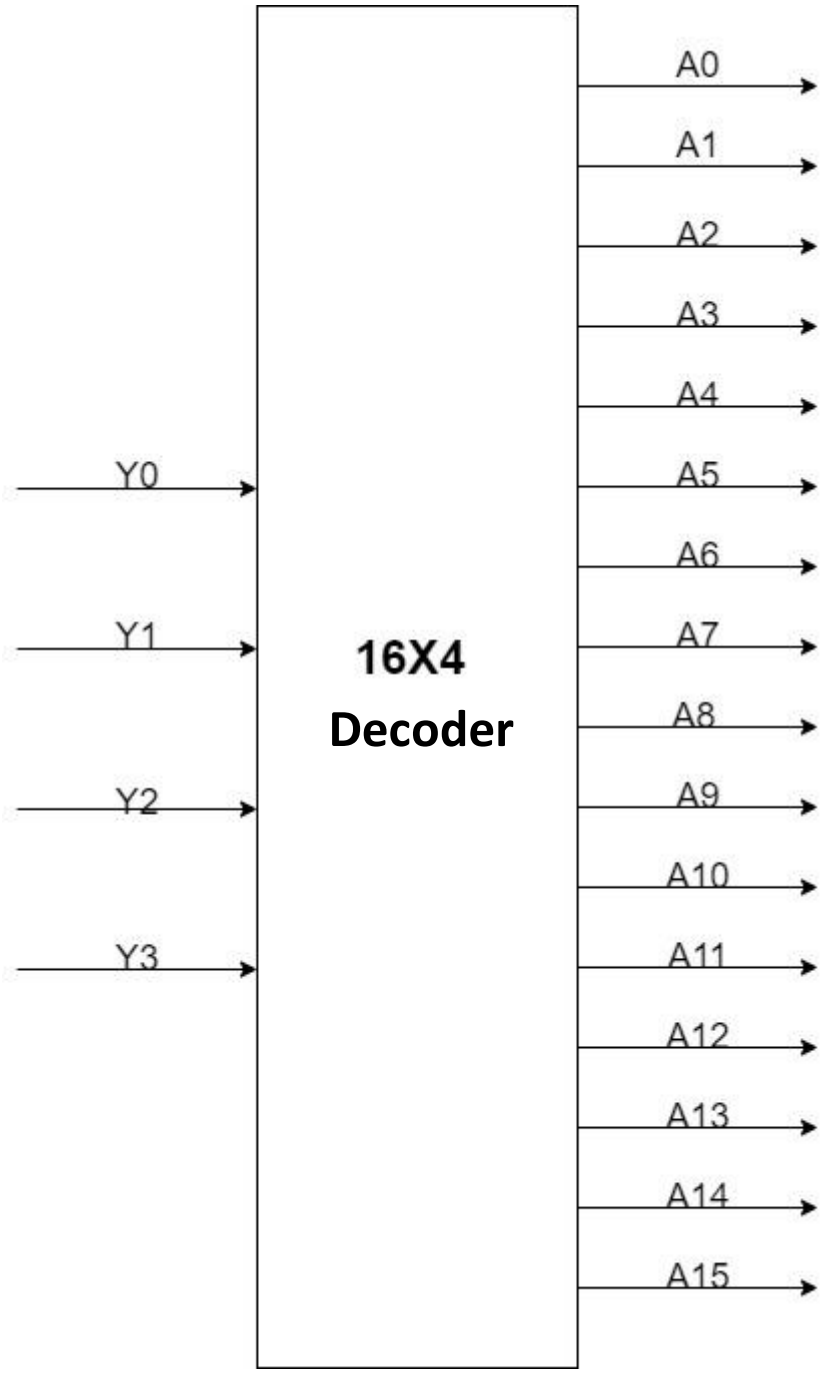

Figure 4 decoder

2.1.4. Truth table for 16X4 Decoder

A_3	A_2	A_1	A_0	Y_{15}	Y_{14}	Y_{13}	Y_{12}	Y_{11}	Y_{10}	Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 5 Decoder truth table

2.2. Simplified expressions for the outputs in encoder and decoder

2.2.1. Encoder simplified outputs

$A_0 = Y_1+Y_3+Y_5+Y_7+Y_9+Y_{11}+Y_{13}+Y_{15}$

$A_1 = Y_2+Y_3+Y_6+Y_7+Y_{10}+Y_{11}+Y_{14}+Y_{15}$

$A_2 = Y_4+Y_5+Y_6+Y_7+Y_{12}+Y_{13}+Y_{14}+Y_{15}$

$A_3 = Y_8+Y_9+Y_{10}+Y_{11}+Y_{12}+Y_{13}+Y_{14}+Y_{15}$

2.2.2. Decoder simplified outputs

$Y_0 = A_0'.A_1'.A_2'.A_3'$

$Y_1 = A_0'.A_1'.A_2'.A_3$

$Y_2 = A_0'.A_1'.A_2.A_3'$

$Y_3 = A_0'.A_1'.A_2.A_3$

$Y_4 = A_0'.A_1.A_2'.A_3'$

$Y_5 = A_0'.A_1.A_2'.A_3$

$Y_6 = A_0'.A_1.A_2.A_3'$

$Y_7 = A_0'.A_1.A_2.A_3$

$Y_8 = A_0.A_1'.A_2'.A_3'$

$Y_9 = A_0.A_1'.A_2'.A_3$

$Y_{10} = A_0.A_1'.A_2.A_3'$

$Y_{11} = A_0.A_1'.A_2.A_3$

$Y_{12} = A_0.A_1.A_2'.A_3'$

$Y_{13} = A_0.A_1.A_2'.A_3$

$Y_{14} = A_0.A_1.A_2.A_3'$

$Y_{15} = A_0.A_1.A_2.A_3$

2.3. Simulated circuits for the encoder and decoder

2.3.1. Encoder

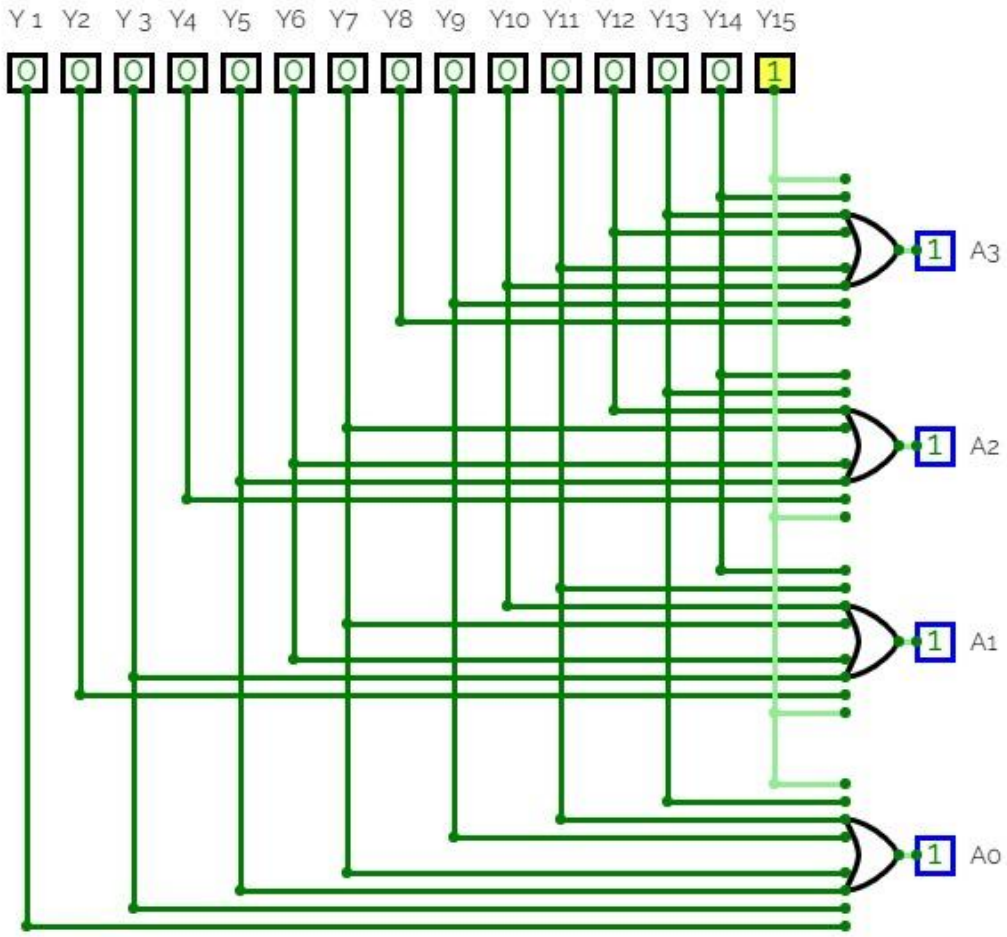

Figure 5Simulated encoder

2.3.2. Decoder

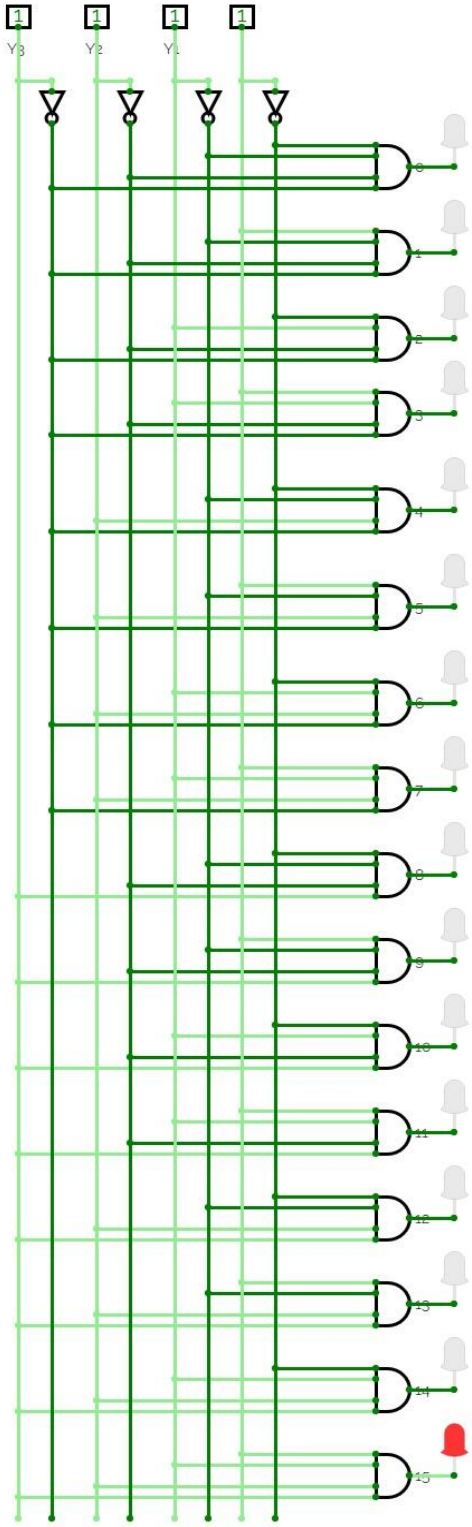

Figure 6 Simulated decoder

3. Question 3

3.1. A Full Adder can be implemented using multiple Half Adders. Using a suitable simulation, show how this could be achieved and explain how it works.

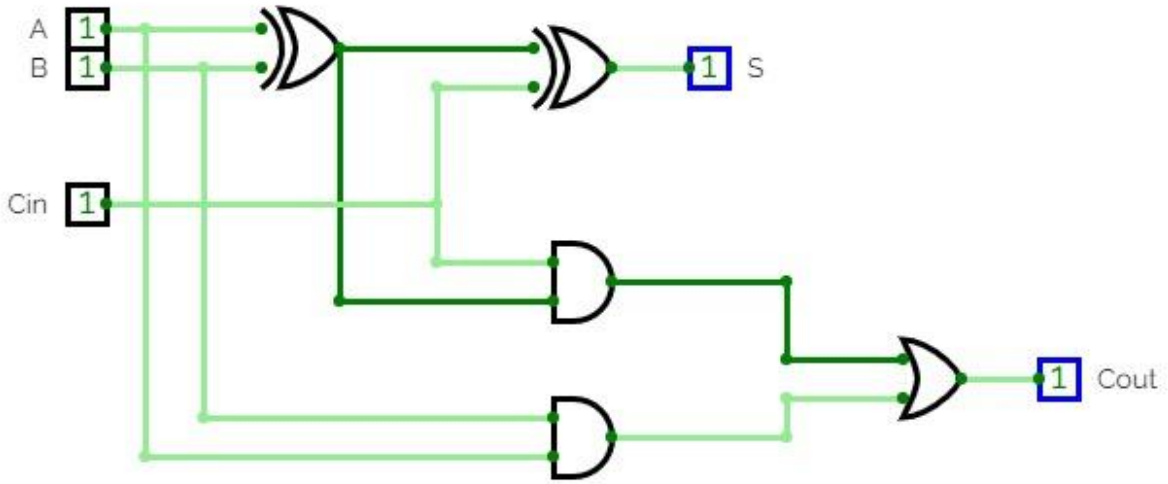

Figure 7

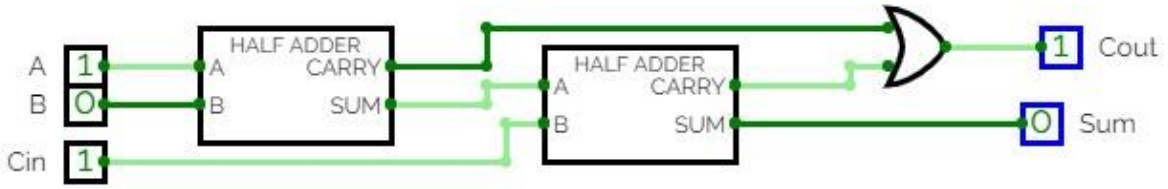

Figure 8

A half adder is a digital circuit that can add two single-bit binary numbers and produce a sum and a carry bit. To implement a full adder using half adders, we need to combine two half adders to create a circuit that can add three single-bit binary numbers: two inputs and a carry-in bit. A full adder is constructed using 2 half adders by using an OR gate to combine the carry-out bits.

3.2. A set of Full Adders can be used to implement an adder which can add two 6-bit binary numbers. These types of adders are also called 'Ripple Carry Adders'. Using a suitable simulation, show how this could be achieved and explain how it works.

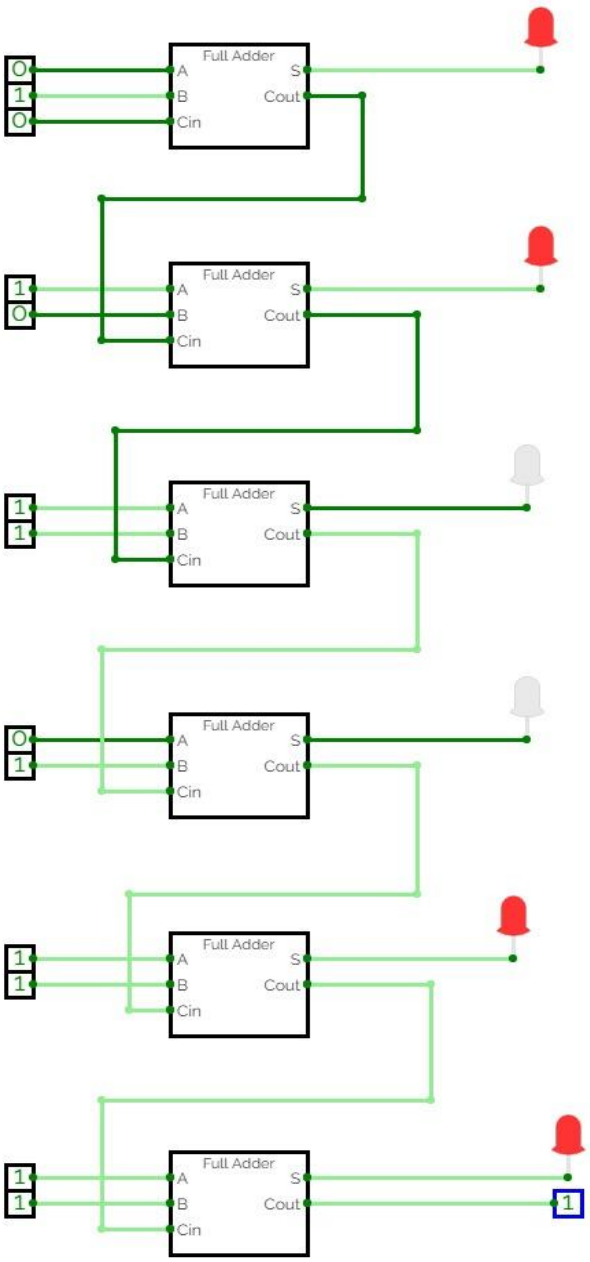

Figure 9 Ripple carry adder

A Full Adder is a digital circuit that performs the addition of three bits: A, B, and Carry-in (Cin) and produces two outputs: a sum (S) bit and a carry-out (Cout) bit. A set of Full Adders can be used to implement an adder that can add two 6-bit binary numbers, which is called a Ripple Carry Adder.

The Ripple Carry Adder works by adding the bits of the two binary numbers one by one, starting from the least significant bit (LSB) and propagating the carry generated from each Full Adder to the next Full Adder until the most significant bit (MSB) is reached. The carry-out generated from the LSB Full Adder is used as the carry-in for the next Full Adder in the sequence. We can construct it using 6 full adders.

4. Workload Matrix

Task Breakdown	20221044	20221041	20221034	20221125	20221097
Question 1	✓	✓	✓	✓	✓
Question 2	✓				
Question 3		✓			✓
Writing the report content	✓	✓	✓	✓	
Editing and formatting the document	✓				
Circuits simulation and drawing	✓	✓		✓	✓